

COMPSCI 701:
Creating Maintainable Software

Lecture 11.2: Case Studies — TDD and Encapsulating Collections

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Agenda

- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

- Admin
 - Noughts and Crosses implementation
 - Questions?
- Case Study in TTD
- Case Study in improving design

Previously in COMPSCI701

- Agenda
- **Review**
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

- Creating software involves many decisions
- It would be nice to know the right decision to make all the time
- Some decisions can be guided by heuristics, principles, and advice
- But some decisions are difficult to make even with such guidance
- What is needed is a **process** that usually (or at least often) helps make a good decision
- One candidate for such a process is **test-driven development**
 - development is guided by **already-written** tests
 - tests must be able to be executed $\Rightarrow$ design must support testing
 - So, design must be **testable**

Test-Driven Development

- Agenda
- Review
- **TDD**
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

- According to Kent Beck (*Test-Driven Development: By Example*, Pearson 2003):
 1. Add a little test — one that causes the existing implementation to fail
 2. Run all tests and fail (on the new one)
 3. Make a little change — the minimum required to get the test to pass, don't worry about how “good” the result is
 4. Run the tests and succeed (all, including new one)
 5. Refactor to remove duplication
- According to Martin, three laws:
 1. You may not write production code until you have written a failing unit test
 2. You may not write more of a unit test than is sufficient to fail, and not compiling is failing
 3. You may not write more production code than is sufficient to pass the currently failing test.

(Implied — keep code, including test code, clean)

TDD Notes

- Agenda
- Review
- TDD
- **TDD Notes**
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

- **TDD is not about creating good tests!**
- All the rules about maintainable code apply to test code (and how it relates to the code being tested), especially:
 - comprehensibility — what functionality does this test? which test tests this piece of code?
 - alterability — if I change this code, which tests need to change?
- TDD doesn't give us design — experience and knowledge does that
<https://blog.cleancoder.com/uncle-bob/2017/03/03/TDD-Harms-Architecture.html>

- Agenda
- Review
- TDD
- TDD Notes
- **TDD Example**
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

TDD Example: Rainfall

- Given an array of integers return the average of all the positive integers in the array that are before the magic constant of -999. Return -1 if the array is null or there are no positive integers.

- Agenda
- Review
- TDD
- TDD Notes
- **TDD Example**
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

TDD Example: Rainfall

```
@Test
public void testSingleValid() {
    int[] input = { 2 };
    double expected = 2.0;
    double actual = Rainfall.rainfall(input);
    assertEquals(expected, actual, 0.00001);
}

// All tests pass, yay!
public static double rainfall(int[] input) {
    return 2.0;
}
```

- Note: When the test was first written, it did not compile because the `rainfall()` method did not exist (so test failed)

- Agenda
- Review
- TDD
- TDD Notes
- **TDD Example**
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

The next 4 tests

- Next 4 iterations deal with each of these tests in order
- Also refactored tests to make it easier to see what was going on in them

```
@Test
public void testOneValidResultNot2() {
    runTest(new int[] {1}, 1.0);
}

@Test
public void testEmpty() {
    runTest(new int[] {}, -1.0);
}

@Test
public void testOneInvalid() {
    runTest(new int[] { -7 }, -1.0);
}

@Test
public void testSentinelOnly() {
    runTest(new int[] { 99999 }, -1.0);
}
```


- Agenda
- Review
- TDD
- TDD Notes
- **TDD Example**
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

After Iteration 5

- Implementation that passes all 5 tests so far!
- No loops (yet)

```
public static double rainfall(int[] input) {  
    if (input.length == 0) {  
        return -1.0;  
    }  
    if (input[0] < 0) {  
        return -1.0;  
    }  
    if (input[0] == 99999) {  
        return -1.0;  
    }  
    return input[0];  
}
```

Other tests that pass

- All of these tests pass with the current iteration!
- From a testing point of view, these are all testing different things and so are needed
- From a TDD point of view, they are unnecessary (since TDD isn't about testing)

```
runTest(new int[] {99999, 5}, -1.0);
runTest(new int[] {99999, 13, 19, 15, 7}, -1.0);
runTest(new int[] {99999, -2}, -1.0);
runTest(new int[] {99999, -2, -999}, -1.0);
runTest(new int[] {99999, -2, 7, 99, -33}, -1.0);
runTest(new int[] {15, 99999}, 15.0);
runTest(new int[] {473, 99999, 2}, 473.0);
runTest(new int[] {33, 99999, 2}, 33.0);
runTest(new int[] {501, 99999, -1}, 501.0);
runTest(new int[] {713, 99999, -399, -3}, 713.0);
runTest(new int[] {444, 99999, 13, -1, -15}, 444.0);
runTest(new int[] {-13, 99999}, -1.0);
runTest(new int[] {-541, 99999, -5}, -1.0);
runTest(new int[] {-22, 99999, 10}, -1.0);
runTest(new int[] {15, -35}, 15.0);
runTest(new int[] {473, -399, 2}, 473.0);
runTest(new int[] {33, -193, 2}, 33.0);
runTest(new int[] {501, -2, -1}, 501.0);
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Remove duplication

```
public static double rainfall(int[] input) {  
    if (input.length == 0 || input[0] < 0 || input[0] == 99999) {  
        return -1.0;  
    }  
    return input[0];  
}
```

- Remove duplication—“return -1.0;”

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Next Tests

```
runTest(new int[] { 5, 99999 }, 5.0);
runTest(new int[] { 5, 5 }, 5.0);

public static double rainfall(int[] input) {
    if (input.length == 0 || input[0] < 0 || input[0] == 99999) {
        return -1.0;
    }
    if (input.length == 1) {
        return input[0];
    }
    if (input[1] == 99999) {
        return input[0];
    }
    return (input[0] + input[1])/2.0;
}
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Remove Duplication

```
public static double rainfall(int[] input) {  
    if (input.length == 0 || input[0] < 0 || input[0] == 99999) {  
        return -1.0;  
    }  
    if (input.length == 1) {  
        return input[0]/1.0;  
    }  
    if (input[1] == 99999) {  
        return input[0]/1.0;  
    }  
    return (input[0] + input[1])/2.0;  
}
```

- Identifying “duplication” is not always easy.

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Remove Duplication next step

```
public static double rainfall(int[] input) {  
    if (input.length == 0 || input[0] < 0 || input[0] == 99999) {  
        return -1.0;  
    }  
    double numValid = input.length;  
    if (input.length == 1) {  
        return input[0]/numValid;  
    }  
    if (input[1] == 99999) {  
        numValid = 1.0;  
        return input[0]/numValid;  
    }  
    return (input[0] + input[1])/numValid;  
}
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Removing duplication with a loop

```
public static double rainfall(int[] input) {  
    if (input.length == 0 || input[0] < 0 || input[0] == 99999) {  
        return -1.0;  
    }  
    int numValid = input.length;  
    if (input.length == 1) { // This was removed  
        return input[0]/numValid;  
    }  
    if (input.length > 1 && input[1] == 99999) {  
        numValid = 1;  
        return input[0]/(double)numValid;  
    }  
    int total = 0;  
    for (int value: input) {  
        total += value;  
    }  
    return total/(double)numValid;  
}
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

More unnecessary tests

```
runTest(new int[] { 3, 3, 3 }, 3.0);  
runTest(new int[] { 1, 3, 2 }, 2.0);  
runTest(new int[] { 1, 2, 3, 4, 5 }, 3.0);  
runTest(new int[] { 5, 10, 16, 9, 1 }, 8.2);
```


- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Next Test: Invalid values

```
runTest(new int[] { 3, -3 }, 3.0);

public static double rainfall(int[] input) {
    if (input.length == 0 || input[0] < 0 || input[0] == 99999) {
        return -1.0;
    }
    int numValid = input.length;
    if (input.length > 1 && input[1] == 99999) {
        numValid = 1;
    }
    int total = 0;
    int validCount = 0;
    for (int i = 0; i < numValid; i++) {
        if (input[i] >= 0) {
            total += input[i];
            validCount++;
        }
    }
    if (validCount > 0) {
        return total/(double)validCount;
    } else {
        return -1.0;
    }
}
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Next Test: Sentinels

```
runTest(new int[] { 5, 5, 5, 99999 }, 5.0);
```

```
public static double rainfall(int[] input) {
    if (input.length == 0 || input[0] == 99999) {
        return -1.0;
    }
    int numValid = input.length;
    if (input.length > 1 && input[1] == 99999) {
        numValid = 1;
    }
    int total = 0;
    int validCount = 0;
    for (int i = 0; i < numValid; i++) {
        if (input[i] == 99999) {
            break;
        }
        if (input[i] >= 0) {
            total += input[i];
            validCount++;
        }
    }
    if (validCount > 0) {
        return total/(double)validCount;
    } else {
        return -1.0;
    }
}
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Removing Duplication (and smells)

```
public static double rainfall(int[] input) {
    if (input.length == 0 || input[0] == 99999) {
        return -1.0;
    }
    int numValid = input.length;
    if (input.length > 1 && input[1] == 99999) {
        numValid = 1;
    }
    double total = 0;
    int validCount = 0;
    for (int i = 0; i < input.length; i++) {
        if (input[i] == 99999) {
            break;
        }
        if (input[i] >= 0) {
            total += input[i];
            validCount++;
        }
    }
    if (validCount > 0) {
        return total/(double)validCount;
    } else {
        return -1.0;
    }
}
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- **TDD Example**
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Final(?)

```
public static double rainfall(int[] input) {  
    double total = 0;  
    int validCount = 0;  
    for (int i = 0; i < input.length; i++) {  
        if (input[i] == 99999) {  
            break;  
        }  
        if (input[i] >= 0) {  
            total += input[i];  
            validCount++;  
        }  
    }  
    if (validCount > 0) {  
        return total/validCount;  
    } else {  
        return -1.0;  
    }  
}
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Evaluation of TDD

- TDD $\neq$ testing — doing proper testing requires more test cases
- testing at the “API” level
- determining what the next test is seems non-obvious — may need to add tests that pass to identify a “small failing test”
- determining the “simplest change” to pass the tests is not always clear — experience will affect what “simplest” means
- “remove duplication”
 - can be difficult to detect what duplication there is — what constitutes “duplication”?
 - “duplication” includes “redundant/unnecessary” and “data duplication”
 - seems to be where all the “good engineering” is done
 - can be difficult to determine how to remove it
 - unclear what the consequences are if you get it wrong
 - but “backing out” is considered a reasonable option
- Questions:
 - To what degree does the order the tests are developed impact the design?
 - Does TDD rule out certain (good) designs?
 - E.g. Could the filter-then-process design for rainfall come from TDD? (loops are duplication?)

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

The Research

- D. Janzen and H. Saiedian, “Does Test-Driven Development Really Improve Software Design Quality?,” in *IEEE Software*, vol. 25, no. 2, pp. 77-84, March-April 2008, <https://doi.org/10.1109/MS.2008.34>
 - Mix of student and industry teams, with varying levels of control
 - TDD teams **tended** to write smaller classes and methods
 - “Complexity” lower for TDD teams
 - “Coupling” (CBO) lower for TDD
 - “Cohesion” (LCOM5) no obvious trend
 - **Differences were much smaller for industry teams than student teams**

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Remember Code Smells

- Code smell - characteristic of design that indicate potential code or design quality (usually maintainability) problems
- *Heuristics* — not hard and fast rules but just an indication that there may be a problem and so requires a closer look
- Remove design smells by applying refactoring (assuming you decide they need to be removed)
- From Martin Fowler “Refactoring: Improving the design of existing code” Addison-Wesley, 1999 (Design Smell chapter co-written with Kent Beck)

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

The problem with heuristics

- Duplicated Code
- Long Method
- Large Class
- Long Parameter List
- Speculative Generality — adding parameters or other ways to generalise a class/method/design without the actual need for the generality
- Data class — almost no methods other than getters and setters for fields
- Comments as deodorant — explaining in comments something that could be made more clear in code by refactoring the code

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

The problem with heuristics

- Duplicated Code — how much duplication? how to remove it?
- Long Method — how long is too long? what code to remove
- Large Class — how long is too long? which methods to remove?
- Long Parameter List — how long is too long? which parameters to remove
- Speculative Generality — adding parameters or other ways to generalise a class/method/design without the actual need for the generality — what if it's easy to do?
- Data class — almost no methods other than getters and setters for fields — what if I know I'll need more methods later?
- Comments as deodorant — explaining in comments something that could be made more clear in code by refactoring the code — ok so I'll just not put in any comments in and that will fix it right?
- One difficulty with heuristics is knowing when they apply
- Another difficulty is knowing exactly what to do

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- **Unencaps. Collection**
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Design Smell: Unencapsulated Collection

```
public interface Book {  
    String getSeries();  
    String getAuthor();  
    String getTitle();  
}
```

```
public static void main(String[] args) {  
    List<Book> library = loadBooks(); // sorted by author  
    System.out.println("There are " + library.size() +  
        " books in the collection");  
    String authorSearch = "John Ronald Reuel Tolkien";  
    System.out.println("Books by '" + authorSearch + "' are:");  
    for (Book book: library) {  
        code omitted  
    }  
    String seriesSearch = "The Lord of the Rings";  
    System.out.println("Books in series '" + seriesSearch + "' are:");  
    code omitted  
    String keywordSearch = "Lord";  
    System.out.println("Books with the word '" + keywordSearch +  
        "' in their details are:");  
    code omitted #1  
}
```

Unencapsulated Collection

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

omitted code #1

```
String keywordSearch = "Lord";
System.out.println("Books with the word '" + keywordSearch +
    "' in their details are:");
for (Book book: library) {
    if (book.getAuthor().indexOf(keywordSearch) >= 0 ||
        book.getTitle().indexOf(keywordSearch) >= 0 ||
        book.getSeries().indexOf(keywordSearch) >= 0) {
        System.out.println("\t" + book.getTitle() + "' by " +
            book.getAuthor() + " (series '" +
            book.getSeries() + "')");
    }
}
```

Client Cohesion

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

```
public static void main(String[] args) {
    List<Book> library = loadBooks(); // sorted by author
    System.out.println("There are " + library.size() +
        " books in the collection");
    String authorSearch = "John Ronald Reuel Tolkien";
    System.out.println("Books by '" + authorSearch + "' are:");
    for (Book book: library) {
        code omitted
    }
    String seriesSearch = "The Lord of the Rings";
    System.out.println("Books in series '" + seriesSearch + "' are:");
    code omitted
    String keywordSearch = "Lord";
    System.out.println("Books with the word '" + keywordSearch +
        "' in their details are:");
    code omitted #1
}
```

- How long is this method?
- What else might it have to do?

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- **Unencaps. Collection**
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Unencapsulated Collection

- Often the context schema involves a collection of items (deck of cards, books, students, etc)
- Usually we do not expect to use the full set of collection methods on our collection (e.g. deck of cards should, once created, never change; would never expect to use `toArray()`)
- Representing these context-schema-specific collections with general-purpose collections (e.g. `java.util.List`) means:
 - the collection can be changed in ways inconsistent with the context schema — a possible **source of faults**
 - client code (that uses the collection) must provide the code to do any necessary processing — possibly leading to **duplication**
 - client **cohesion** is reduced
 - the implementation decision to represent the context schema collection with a general purpose collection is **not hidden**
 - **⇒ no encapsulation**
 - Such a use of a collection is called an **unencapsulated collection**—See G. De Ruvo, E. Tempero, A. Luxton-Reilly and N. Giacaman 'Unencapsulated Collection — A Teachable Design Smell' *SIGCSE Technical Symposium*, February 2018. pp. 332-337. <https://doi.org/10.1145/3159450.3159469>
- So, **Encapsulate the collection**

Unencapsulated Collection Smell Identification

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

- General Collections (`List`, `Collection`, `ArrayList`, arrays, etc) returned by methods, used as types of fields, and so on

```
public static void main(String[] args) {  
    List<Book> library = loadBooks(); // sorted by author  
    System.out.println("There are " + library.size() +  
        " books in the collection");  
    String authorSearch = "John Ronald Reuel Tolkien";  
    System.out.println("Books by '" + authorSearch + "' are:");  
    ....  
}
```

- These collections are used in a **context schema-specific way**—look at where and how they are used

```
omitted code #1  
String keywordSearch = "Lord";  
System.out.println("Books with the word '" + keywordSearch + "' in their details are:");  
for (Book book: library) {  
    if (book.getAuthor().indexOf(keywordSearch) >= 0 ||  
        book.getTitle().indexOf(keywordSearch) >= 0 ||  
        book.getSeries().indexOf(keywordSearch) >= 0) {  
        System.out.println("  " + book.getTitle() + "' by " +  
            book.getAuthor() + " (series '" +  
            book.getSeries() + "')");  
    }  
}
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- **Encapsulating Collections**
- Refinement
- Further refinement
- Key Points

Encapsulating Collections

1. Create a class to represent the **context schema-specific collection**
2. Make the unencapsulated collection a field of the new class
3. Move the (relevant parts of the) code that does the processing of the list into methods for the new class
4. Adjust the client code to use the new class

⇒ refactoring

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- **Encapsulating Collections**
- Refinement
- Further refinement
- Key Points

Steps 1–3

```
public class BookCollection {
    private List<Book> _library;
    public BookCollection() {
        _library = loadBooks();
    }
    public void reportCollectionSize() {
        System.out.println("There are " + _library.size() +
            " books in the collection");
    }
    public void authorSearch(String authorSearch){ code omitted }
    public void listSeries(String seriesSearch){ code omitted }
    public void keywordSearch(String keywordSearch) {
        System.out.println("Books with the word '" + keywordSearch +
            "' in their details are:");
        for (Book book: _library) {
            if (book.getAuthor().indexOf(keywordSearch) >= 0 ||
                book.getTitle().indexOf(keywordSearch) >= 0 ||
                book.getSeries().indexOf(keywordSearch) >= 0) {
                System.out.println("\t" + book.getTitle() + "' by " +
                    book.getAuthor()+" (series '"+book.getSeries()+"'");
            }
        }
    }
}
```

- cohesion is improved $\Rightarrow$ comprehensibility
- encapsulation means the collection cannot be changed in un-anticipated ways $\Rightarrow$ comprehensibility
- N has increased, changes to the collection management will mostly be limited to `BookCollection` $\Rightarrow \mathcal{CR}$ reduced — alterability
- testing can be done independent of the client code $\Rightarrow$ testability

Step 4

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- **Encapsulating Collections**
- Refinement
- Further refinement
- Key Points

```
public static void main(String[] args) {  
    BookCollection myCollection = new BookCollection();  
    myCollection.reportCollectionSize();  
    String authorSearch = "John Ronald Reuel Tolkien";  
    myCollection.authorSearch(authorSearch);  
    String seriesSearch = "The Lord of the Rings";  
    myCollection.listSeries(seriesSearch);  
    String keywordSearch = "Lord";  
    myCollection.keywordSearch(keywordSearch);  
}
```

- smaller (and reasonable method names) $\Rightarrow$ comprehensibility

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- **Refinement**
- Further refinement
- Key Points

Refining Abstraction

- After the refactoring comes the refinement

```
public void keywordSearch(String keywordSearch,
    boolean inAuthor, boolean inTitle, boolean inSeries) {
    String location = "in their details are: ";
    if (inAuthor) {
        if (!inTitle || !inSeries) { location = "in author are: "; }
    }
    if (inTitle) {
        if (!inAuthor || !inSeries) { location = "in the title are: "; }
    }
    if (inSeries) {
        if (!inTitle || !inAuthor) { location = "...etc...: "; }
    }
    System.out.println("Books with the word '" + keywordSearch + "' " +
        location);
    for (Book bk: _library) { // bad name to fit in space
        if ((inAuthor && bk.getAuthor().indexOf(keywordSearch) >= 0) ||
            (inTitle && bk.getTitle().indexOf(keywordSearch) >= 0) ||
            (inSeries && bk.getSeries().indexOf(keywordSearch) >= 0)) {
            System.out.println("\t" + bk.getTitle() + " by " +
                bk.getAuthor() + " (series '" + bk.getSeries() + "')");
        }
    }
}
```

Refinement

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- **Refinement**
- Further refinement
- Key Points

```
public static void main(String[] args) {  
    BookCollection myCollection = new BookCollection();  
    myCollection.reportCollectionSize();  
    String authorSearch = "John Ronald Reuel Tolkien";  
    myCollection.authorSearch(authorSearch);  
    String seriesSearch = "The Lord of the Rings";  
    myCollection.listSeries(seriesSearch);  
    String keywordSearch = "Lord";  
    myCollection.keywordSearch(keywordSearch, false, true, false);  
}
```

- technically not refactoring because functionality has changed
- but harder to see how to make this change in functionality when dealing with masses of code
- now thinking of what the `BookCollection` abstraction should be, rather than how do manage a list of `Book`

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- **Further refinement**
- Key Points

Further refinement

```
public class BookCollection implements Iterable<Book> {  
    private List<Book> _library;  
    public BookCollection() {  
        _library = loadBooks();  
    }  
  
    public Iterator<Book> iterator() {  
        return _library.iterator();  
    }  
    ....  
}
```

- Agenda
- Review
- TDD
- TDD Notes
- TDD Example
- TDD Example
- TDD Evaluation
- The Research
- Smells
- Unencaps. Collection
- Encapsulating Collections
- Refinement
- Further refinement
- Key Points

Key Points

- TDD can lead to successful designs I would not have necessarily have chosen
- Various questions remain as to the cost/benefit tradeoff, but I am more inclined to believe than disbelieve
- But lots of research opportunities
- A common poor design decision (aka “design smell”) is to pass around a collection of items and then processing the collection in different ways — an unencapsulated collection
- Remove smell by refactoring to encapsulate collection by creating a class with the collection as a field, and methods in the class do the different processing that is required.